Assignment 06: Passwords and Hash Tables

Talan Jewett | CS 210 | San Diego State University | October 18, 2026

GitHub repository	https://github.com/YOUR-USERNAME/cs210-assignment06
Submitted version	tag **v1.0-submission**
Build	`g++ -std=c++17 -O2 main.cpp -o a06`, then `./a06` (demo), `./a06 test`, `./a06 bench`. Zero warnings with -Wall -Wextra -Wpedantic.
Result	Demo: 28/28 checks pass. Tests: 233/233 assertions pass on all four strategies. Clean under AddressSanitizer + UndefinedBehaviorSanitizer.

AI use disclosure. I used AI (Claude) to help with the formatting of the code and PDF documents. I also used it to help draft and test the implementation, benchmark, and write-up, and I reviewed and ran all of it myself.

1. Overview

The program is a small authentication system whose user store is a hash table I wrote from scratch (no `std::unordered_map`). Keys are usernames; values are `UserRecord`s holding a per-user salt and `demoHash(salt + password + pepper)`. The plaintext password is never stored.

I implemented four collision strategies behind one interface (`UserTable`): **separate chaining**, **linear probing**, **quadratic probing**, and **double hashing**. Each supports `insert`, `find`, `remove`, `loadFactor`, and automatic resize/rehash. After benchmarking all four (Section 3), I chose **separate chaining**; that class is named `HashTable` and is the one the main demo uses.

2. Design decisions

- **One interface, four implementations.** `AuthService` receives a `std::unique_ptr<UserTable>`, so the identical login code, tests and benchmark run against every strategy. That is what makes the comparison fair.
- **Power-of-two capacities.** The bucket index is a bit mask instead of `%`. It also guarantees full coverage: triangular quadratic probing (*home* + *i(i+1)/2*) and double hashing with an *odd* step both visit every slot of a 2^k table, so probing can never loop forever while a free slot exists.
- **Folding the hash.** `homeIndex = (h ^ (h >> 32)) & (cap - 1)` lets the high bits of the 64-bit hash influence small tables. Double hashing's step uses a separate Fibonacci-multiplied mix forced odd.
- **Cached hashes.** Every node/slot stores its 64-bit hash. Rehashing never recomputes `demoHash`, and most non-matching entries are rejected by one integer compare before any string compare.
- **Resize policy.** Chaining doubles when an insert would push the load factor above 0.75. Open addressing doubles above 0.5, and counts tombstones toward that limit because they lengthen probe sequences just like live keys. If a table is mostly tombstones it is rebuilt at the *same* size instead of growing.
- **Deletion.** Chaining simply unlinks the node. Open addressing marks the slot *Deleted* (a tombstone): marking it *Empty* would cut the probe chain and make keys further along it unfindable. A test removes a key from the middle of a six-key collision cluster to prove this works.

- **Pointer stability.** Chaining's rehash relinks existing nodes instead of copying them, so a pointer returned by `find()` stays valid across a resize.
- **Salt and pepper.** Each user gets a 64-bit random salt (16 hex chars), so identical passwords produce different hashes (gabe and hiro both use *hunter2* in the demo). The pepper is one secret shared by all accounts and kept out of the table. The salt generator is seeded with a fixed value only so the output in this PDF is reproducible.

3. Comparing the collision strategies

Part A: average probes per lookup at fixed load factors (capacity 65,536, resizing disabled). A probe is one slot or chain node examined; an empty chaining bucket counts as one probe.

Strategy	hit .25	miss .25	hit .50	miss .50	hit .75	miss .75	hit .90	miss .90
separate chaining	1.12	1.03	1.25	1.10	1.38	1.23	1.45	1.32
linear probing	1.17	1.38	1.50	2.45	2.49	8.53	5.58	59.94
quadratic probing	1.16	1.36	1.43	2.15	1.99	4.69	2.88	12.95
double hashing	1.15	1.33	1.38	2.00	1.85	4.02	2.56	10.03

The measurements match the textbook formulas closely, which is good evidence the probe sequences are implemented correctly: at load 0.9, linear probing's predicted successful/unsuccessful costs are 5.5 / 50.5 (measured 5.58 / 59.94), double hashing's (ideal uniform hashing) are 2.56 / 10.0 (measured 2.56 / 10.03), and chaining's successful cost $1 + \alpha/2 = 1.45$ (measured 1.45).

Part B: wall-clock time for 200,000 users with default load limits, starting at capacity 8, best of 3 runs (ms).

Strategy	insert	find hit	find miss	remove 50%	re-insert	final cap	rehashes
separate chaining	31.0	7.7	6.7	5.6	11.1	524288	16
linear probing	93.7	17.1	15.2	8.7	104.3	1048576	17
quadratic probing	95.0	17.6	16.9	9.0	103.7	1048576	17
double hashing	103.4	15.7	17.0	9.6	108.8	1048576	17

Timings are machine-dependent; the relative pattern is what matters.

What the numbers show

- **Linear probing** suffers *primary clustering*: occupied runs merge into long blocks, so a miss at load 0.9 costs ~60 probes.
- **Quadratic probing** breaks up primary clusters but keys with the same home slot still follow the same path (*secondary clustering*): ~13 probes per miss at 0.9.
- **Double hashing** gives each key its own step size, so it behaves like ideal uniform hashing and is the best of the open-addressing schemes.
- **Separate chaining** uses the fewest probes at every load factor and degrades gently because a collision only lengthens one short list. It was also fastest in Part B. Part of that is specific to my implementation: each open-addressing slot embeds three `std::string`s (120 bytes), and at max load 0.5 the table holds at least two slots per user, while chaining allocates one 120-byte node per user plus an 8-byte bucket pointer. With small fixed-size keys, linear probing's cache locality often wins at low load, so I do not claim chaining is always faster.
- **Deletes hurt open addressing.** Tombstones count toward the load limit, which is why the re-insert phase triggered an extra rehash (final capacity 1,048,576 vs 524,288 for chaining).

Choice: separate chaining

I chose separate chaining for the final `HashTable` because it had the lowest and most predictable probe counts, the simplest correct deletion (no tombstones, which matters for an account system where users are removed), pointers that survive rehashing, and it stayed efficient even if the load factor is allowed to exceed 1. If memory locality mattered more than deletes, double hashing would be my open-addressing choice.

4. Concepts

Collisions

A collision happens when two different keys map to the same bucket. They are unavoidable: `demoHash` produces 2^{64} values but the table has only a few buckets, so by the pigeonhole principle many keys share each bucket index. By the birthday effect they happen early: in the demo, 8 users in 16 buckets produced a 3-long chain (gabe -> carmen -> alice in bucket 9) and a 2-long chain (dmitri -> eunji in bucket 4). A collision strategy decides where the second key goes.

Chaining

In separate chaining each bucket holds a linked list of every entry that hashed there. `insert` scans that one list for a duplicate, then pushes a new node on the front; `find` walks the list comparing cached hashes then keys; `remove` unlinks the matching node. Only keys in the same bucket are ever compared.

Load factor

The load factor is $\alpha = n / m$ (entries / buckets). In chaining it equals the average chain length. In open addressing it is the fraction of slots in use and must stay below 1. Keeping α bounded is what keeps operations fast, so the table doubles its capacity and rehashes every entry into the new array when α would exceed the limit (0.75 for chaining). The demo shows three rehash events: 4 -> 8, 8 -> 16 and 16 -> 32 buckets.

Expected lookup complexity

Assuming the hash spreads keys roughly uniformly, a chaining lookup examines an expected $1 + \alpha/2$ nodes when the key is present and about α when it is absent. Because resizing keeps $\alpha \leq 0.75$, both are bounded by a constant, so **find, insert and remove run in expected O(1)** time (strictly O(L) for a key of length L, since `demoHash` reads every character). The **worst case is O(n)**, when every key lands in one bucket. A single rehash costs O(n), but capacity doubles each time, so the total rehash work over n inserts is at most $n + n/2 + n/4 + ... < 2n$: **amortized O(1) per insert**.

Why a classroom hash is not a production password-hashing scheme

- **It is fast on purpose.** FNV-style hashing takes nanoseconds, so an attacker with a leaked table can test billions of guesses per second on a GPU. Password hashing must be deliberately slow and memory-hard: Argon2id, scrypt, bcrypt, or PBKDF2 with a high work factor.
- **It is not cryptographic.** It is just XOR and multiply, so it has no preimage resistance: given a stored hash, an attacker can construct an input that produces it, and any such input logs in.
- **64 bits is too small.** Even with a perfect function, a birthday search finds collisions in about 2^{32} tries.

- **The rest of this demo is also simplified:** salts come from a seeded `mt19937_64` instead of a cryptographically secure generator (and should be 128+ bits), the pepper is in source code instead of a secrets manager, and the login result reveals whether a username exists. A real system returns one generic "invalid username or password" message and adds rate limiting.

5. Complexity summary

Operation	Chaining (chosen)	Open addressing	Notes
find	expected O(1); worst O(n)	expected O(1) while $\alpha < 1$; worst O(n)	$O(1 + \alpha)$ for chaining
insert	amortized expected O(1)	amortized expected O(1)	includes duplicate check; rehash amortized
remove	expected O(1)	expected O(1), leaves tombstone	tombstones raise later probe counts
loadFactor	O(1)	O(1)	size and capacity are cached
rehash	O(n + m)	O(n + m)	cached hashes: no demoHash recompute
login	expected O(1) + O(L)	expected O(1) + O(L)	one find + one demoHash of the salted password
space	O(n + m), m = Θ(n)	O(m), $m \geq 2n$	α stays in [0.375, 0.75] after growth

6. Testing

Every table test runs on all four strategies. `std::map` appears only as a reference oracle in the randomized test; it never stores user records.

- **Hash regression:** fixed outputs for `demoHash`. I noticed the course's offset basis (1469598103934665603) is one digit shorter than standard FNV-1a's (14695981039346656037), so published FNV-1a test vectors don't apply; I used the function exactly as given.
- **Basics and edge cases:** empty table, missing keys, duplicate insert updates instead of adding, remove twice, empty-string key, exact load factor.
- **Resize:** 5,000 inserts from capacity 2; load never exceeds the limit and every key is still found afterward.
- **Forced collisions:** six keys chosen to share bucket 3; remove the middle one and confirm the rest are still reachable.
- **Tombstone churn:** 20,000 insert/remove cycles; the table neither grows without bound nor loops.
- **Randomized differential test:** 50,000 random insert/remove/find operations compared against the reference map.
- **Authentication:** success, wrong password (wrong case, empty), unknown user, duplicate registration, unique salts, removal.

Test output

```
Running hash table tests
  ok    demoHash reference values
  ok    insert/find/remove/loadFactor basics [separate chaining]
  ok    insert/find/remove/loadFactor basics [linear probing]
  ok    insert/find/remove/loadFactor basics [quadratic probing]
  ok    insert/find/remove/loadFactor basics [double hashing]
  ok    automatic resize keeps all keys (5000 inserts) [separate chaining]
```

```
  ok    automatic resize keeps all keys (5000 inserts) [linear probing]
  ok    automatic resize keeps all keys (5000 inserts) [quadratic probing]
  ok    automatic resize keeps all keys (5000 inserts) [double hashing]
  ok    forced collisions + removal mid-cluster [separate chaining]
  ok    forced collisions + removal mid-cluster [linear probing]
  ok    forced collisions + removal mid-cluster [quadratic probing]
  ok    forced collisions + removal mid-cluster [double hashing]
  ok    tombstone churn (20000 insert/remove cycles) [separate chaining]
  ok    tombstone churn (20000 insert/remove cycles) [linear probing]
  ok    tombstone churn (20000 insert/remove cycles) [quadratic probing]
  ok    tombstone churn (20000 insert/remove cycles) [double hashing]
  ok    50000 random ops vs reference map [separate chaining]
  ok    50000 random ops vs reference map [linear probing]
  ok    50000 random ops vs reference map [quadratic probing]
  ok    50000 random ops vs reference map [double hashing]
  ok    auth: register/login/remove [separate chaining]
  ok    auth: register/login/remove [linear probing]
  ok    auth: register/login/remove [quadratic probing]
  ok    auth: register/login/remove [double hashing]

233 assertions passed, 0 failed
```

Demo output (successful login, failed password, unknown user, removal, rehash events)

```
CS 210 Assignment 06 - Passwords and Hash Tables (Talan Jewett)

=== 1. Register 8 users (separate chaining, start capacity 4, max load 0.75) ===
  + alice   salt=936bd9a8143681cd  hash=9c92ab842b1b3eac  load=0.250 (1/4)
  + bob     salt=ad0d30b97f005ee6  hash=90ed8f3be432b383  load=0.500 (2/4)
  + carmen  salt=6493243ba7646169  hash=ed3c1d0ad613cd96  load=0.750 (3/4)
  [REHASH] separate chaining: 4 -> 8 buckets (3 entries relinked)
  + dmitri  salt=8ca5770c11d0cc89  hash=c9ecaaaaf6f3056d  load=0.500 (4/8)
  + eunji   salt=d2f4a0f067ccb678  hash=5472b4a18836ed16  load=0.625 (5/8)
  + farah   salt=583a2c7e696a7cfb  hash=cd18e89e02d609c4  load=0.750 (6/8)
  [REHASH] separate chaining: 8 -> 16 buckets (6 entries relinked)
  + gabe    salt=3eb7e224bb3628bf  hash=9a00bd5d3ca619d8  load=0.438 (7/16)
  + hiro    salt=c217704b436b9f73  hash=f4dca033d9ca1d19  load=0.500 (8/16)
    [PASS] 8 users stored
    [PASS] two automatic rehash events (4->8, 8->16)
    [PASS] load factor stays <= 0.75
    [PASS] duplicate username is rejected

=== 2. Bucket layout (collisions handled by chaining) ===
    [ 0] hiro
    [ 1] (empty)
    [ 2] (empty)
    [ 3] bob
    [ 4] dmitri -> eunji   <-- collision, chain length 2
    [ 5] (empty)
    [ 6] (empty)
    [ 7] (empty)
    [ 8] (empty)
    [ 9] gabe -> carmen -> alice   <-- collision, chain length 3
    [10] (empty)
    [11] farah
    [12] (empty)
    [13] (empty)
    [14] (empty)
    [15] (empty)
    longest chain = 3

=== 3. Salt demonstration: gabe and hiro share the password "hunter2" ===
  gabe: salt=3eb7e224bb3628bf hash=9a00bd5d3ca619d8
  hiro: salt=c217704b436b9f73 hash=f4dca033d9ca1d19
    [PASS] salts differ
    [PASS] same password -> different stored hashes
    [PASS] hash recomputes from stored salt + password + pepper

=== 4. Login flow ===
  login("alice", "Wonderland#42") -> SUCCESS
    [PASS] expected SUCCESS
  login("carmen", "SanDiego2026") -> SUCCESS
    [PASS] expected SUCCESS
  login("alice", "wonderland#42") -> WRONG PASSWORD
    [PASS] expected WRONG PASSWORD
  login("dmitri", "") -> WRONG PASSWORD
    [PASS] expected WRONG PASSWORD
```

```
  login("gabe", "hunter3") -> WRONG PASSWORD
    [PASS] expected WRONG PASSWORD
  login("mallory", "Wonderland#42") -> UNKNOWN USER
    [PASS] expected UNKNOWN USER
  login("Alice", "Wonderland#42") -> UNKNOWN USER
    [PASS] expected UNKNOWN USER

=== 5. Removal ===
    [PASS] remove("bob") returns true
  login("bob", "b0bcat!") -> UNKNOWN USER
    [PASS] expected UNKNOWN USER
    [PASS] removing bob again returns false
    [PASS] find("bob") returns nullptr
    [PASS] size is now 7
  login("eunji", "kimchi&rice") -> SUCCESS
    [PASS] expected SUCCESS
    size=7 capacity=16 load=0.438

=== 6. Rehash event: add users until load would exceed 0.75 ===
  + student1  load=0.500 (8/16)
  + student2  load=0.562 (9/16)
  + student3  load=0.625 (10/16)
  + student4  load=0.688 (11/16)
  + student5  load=0.750 (12/16)
  [REHASH] separate chaining: 16 -> 32 buckets (12 entries relinked)
  + student6  load=0.406 (13/32)
    [PASS] capacity doubled from 16 to 32
    [PASS] third rehash event recorded
    [PASS] every original user still logs in after the rehash
  login("student6", "pw6") -> SUCCESS
    [PASS] expected SUCCESS

=== 7. Same workflow on all four collision strategies ===
  strategy              size   cap    load  rehashes  avg probes  logins
  separate chaining       13    32   0.406         3        1.22  all correct
    [PASS] separate chaining: success / wrong password / unknown user all correct
  linear probing          13    32   0.406         3        1.11  all correct
    [PASS] linear probing: success / wrong password / unknown user all correct
  quadratic probing       13    32   0.406         3        1.11  all correct
    [PASS] quadratic probing: success / wrong password / unknown user all correct
  double hashing          13    32   0.406         3        1.11  all correct
    [PASS] double hashing: success / wrong password / unknown user all correct

Summary: 28 checks passed, 0 failed
```

7. Source code

The whole program is one file, main.cpp (about 1,060 lines), organized into commented sections: hash helpers, the interface, HashTable (chaining), the open-addressing table, the auth service, and the demo, test and bench modes. The same code is embedded in the Canvas post.

main.cpp

```
// main.cpp - CS 210 Assignment 06: Passwords and Hash Tables
// Author: Talan Jewett
//
// Build:  g++ -std=c++17 -O2 -Wall -Wextra main.cpp -o a06
// Run:    ./a06          authentication demo (default)
//         ./a06 test     unit / edge-case / randomized tests on all strategies
//         ./a06 bench    probe-count and timing comparison of the strategies
//
// The user records live in a hand-written hash table (no std::unordered_map).
// Four collision strategies are implemented behind one interface; separate
// chaining (class HashTable) is the one chosen for the authentication demo.

#include <chrono>
#include <cmath>
#include <cstddef>
#include <cstdint>
#include <cstdio>
#include <functional>
#include <iomanip>
#include <iostream>
#include <map>
#include <memory>
#include <random>
#include <string>
#include <utility>
#include <vector>

// ============================================================================
// Hash function and index helpers
// ============================================================================

// demoHash is the course-provided educational hash (FNV-1a style, 64-bit),
// used exactly as given. (Its offset basis is one digit shorter than the
// standard FNV-1a constant 14695981039346656037; it still mixes well.)
// It is fast and well-distributed for hash-table indexing, but it is NOT a
// password-hashing function: it is designed to be fast, which is exactly what
// an attacker guessing passwords wants. See the write-up for details.


inline uint64_t demoHash(const std::string& s) {
    uint64_t h = 1469598103934665603ULL;
    for (unsigned char c : s) {
        h = (h ^ c) * 1099511628211ULL;
    }
    return h;
}

// Smallest power of two >= n (minimum 2). Power-of-two capacities let us use a
// bit mask instead of '%' and guarantee that quadratic (triangular) probing and
// double hashing with an odd step visit every slot.
inline std::size_t roundUpPow2(std::size_t n) {
    std::size_t p = 2;
    while (p < n) p <<= 1;
    return p;
}

// Map a 64-bit hash to a bucket. XOR-folding the upper 32 bits into the lower
// 32 bits means the high bits of the hash still influence the bucket when we
// mask with a small capacity.
inline std::size_t homeIndex(uint64_t h, std::size_t capacity) {
    return static_cast<std::size_t>(h ^ (h >> 32)) & (capacity - 1);
}

// Second hash for double hashing. Multiplying by a large odd constant
// (Fibonacci hashing) decorrelates the step from the home index; forcing the
// result odd makes it coprime with a power-of-two capacity.
inline std::size_t secondaryStep(uint64_t h) {
    return static_cast<std::size_t>((h * 0x9E3779B97F4A7C15ULL) >> 33) | 1u;
}
```

```
inline std::string toHex(uint64_t v) {
    char buf[17];
    std::snprintf(buf, sizeof buf, "%016llx", static_cast<unsigned long long>(v));
    return std::string(buf);
}

// ============================================================================
// User record
// ============================================================================

// One stored account. The plaintext password is never stored: only the
// per-user salt and demoHash(salt + password + pepper).
struct UserRecord {
    std::string username;
    std::string salt;           // 16 hex characters, unique per user
    uint64_t passwordHash = 0;  // demoHash(salt + password + pepper)
};

// ============================================================================
// Common table interface + instrumentation
// ============================================================================

// Common interface for every collision strategy, so the auth service, tests
// and benchmark can run the exact same code against each implementation.



// Instrumentation used by the benchmark to compare strategies.
struct ProbeStats {
    uint64_t lookups = 0;   // number of find() calls
    uint64_t probes = 0;    // slots / chain nodes examined across those calls
    uint64_t rehashes = 0;  // number of resize/rehash events
    double avgProbes() const { return lookups ? static_cast<double>(probes) / lookups : 0.0; }
};

// Called whenever a table rehashes: (strategy, oldCapacity, newCapacity, entriesMoved)
using RehashListener =
    std::function<void(const std::string&, std::size_t, std::size_t, std::size_t)>;

class UserTable {
public:
    virtual ~UserTable() = default;

    // Returns true if the key was new, false if an existing record was replaced.
    virtual bool insert(const std::string& username, const UserRecord& record) = 0;
    // Returns nullptr if the key is absent.
    virtual const UserRecord* find(const std::string& username) const = 0;
    // Returns true if a record was removed.
    virtual bool remove(const std::string& username) = 0;
    // Live entries / capacity.
    virtual double loadFactor() const = 0;
    virtual std::size_t size() const = 0;
    virtual std::size_t capacity() const = 0;
    virtual std::string strategyName() const = 0;

    void setRehashListener(RehashListener listener) { listener_ = std::move(listener); }
    const ProbeStats& stats() const { return stats_; }
    void resetProbeStats() const { stats_.lookups = 0; stats_.probes = 0; }

protected:
    void notifyRehash(std::size_t oldCap, std::size_t newCap) {
        ++stats_.rehashes;
        if (listener_) listener_(strategyName(), oldCap, newCap, size());
    }

    mutable ProbeStats stats_;  // mutable: find() is const but still counts probes
    RehashListener listener_;
};

// ============================================================================
// HashTable: SEPARATE CHAINING (chosen strategy)
// ============================================================================

// HashTable: separate chaining. This is the strategy I chose for the final
// authentication demo (see the write-up for the benchmark that justifies it).
//
// Layout: a power-of-two array of buckets; each bucket is a singly linked list
// of heap-allocated nodes. Each node caches its full 64-bit hash, so rehashing
// never recomputes demoHash and most mismatches are rejected without a string
// comparison.
```

```
class HashTable : public UserTable {
public:
    explicit HashTable(std::size_t initialCapacity = 8, double maxLoad = 0.75)
        : buckets_(roundUpPow2(initialCapacity)), maxLoad_(maxLoad) {}

    ~HashTable() override { clear(); }
    HashTable(const HashTable&) = delete;
    HashTable& operator=(const HashTable&) = delete;

    bool insert(const std::string& username, const UserRecord& record) override {
        const uint64_t h = demoHash(username);
        // Update in place if the key already exists.
        for (Node* n = buckets_[homeIndex(h, buckets_.size())].get(); n; n = n->next.get()) {
            if (n->hash == h && n->key == username) {
                n->value = record;
                return false;
            }
        }
        // Grow before adding if this insert would exceed the max load factor.
        if (static_cast<double>(size_ + 1) / buckets_.size() > maxLoad_) {
            rehash(buckets_.size() * 2);
        }
        auto node = std::make_unique<Node>();
        node->hash = h;
        node->key = username;
        node->value = record;
        std::unique_ptr<Node>& head = buckets_[homeIndex(h, buckets_.size())];
        node->next = std::move(head);  // push-front: O(1)
        head = std::move(node);
        ++size_;
        return true;
    }

    const UserRecord* find(const std::string& username) const override {
        const uint64_t h = demoHash(username);
        ++stats_.lookups;
        std::size_t visited = 0;
        for (const Node* n = buckets_[homeIndex(h, buckets_.size())].get(); n; n = n->next.get()) {
            ++visited;
            if (n->hash == h && n->key == username) {
                stats_.probes += visited;
                return &n->value;
            }
        }
        stats_.probes += visited ? visited : 1;  // inspecting an empty bucket costs one probe
        return nullptr;
    }

    bool remove(const std::string& username) override {
        const uint64_t h = demoHash(username);
        std::unique_ptr<Node>* link = &buckets_[homeIndex(h, buckets_.size())];
        while (*link) {
            if ((*link)->hash == h && (*link)->key == username) {
                *link = std::move((*link)->next);  // unlink; old node is freed
                --size_;
                return true;
            }
            link = &(*link)->next;
        }
        return false;
    }

    double loadFactor() const override {
        return static_cast<double>(size_) / buckets_.size();
    }
    std::size_t size() const override { return size_; }
    std::size_t capacity() const override { return buckets_.size(); }
    std::string strategyName() const override { return "separate chaining"; }

    // ---- Inspection helpers used by the demo to visualise collisions ----
    std::vector<std::string> bucketKeys(std::size_t bucket) const {
        std::vector<std::string> keys;
        for (const Node* n = buckets_[bucket].get(); n; n = n->next.get()) keys.push_back(n->key);
        return keys;
    }
    std::size_t longestChain() const {
        std::size_t best = 0;
        for (std::size_t b = 0; b < buckets_.size(); ++b) {
            std::size_t len = 0;
            for (const Node* n = buckets_[b].get(); n; n = n->next.get()) ++len;
```

```
            if (len > best) best = len;
        }
        return best;
    }

private:
    struct Node {
        uint64_t hash = 0;
        std::string key;
        UserRecord value;
        std::unique_ptr<Node> next;
    };

    // Relink every node into a new bucket array. Nodes are moved, not copied,
    // so pointers returned by find() stay valid across a rehash.
    void rehash(std::size_t newCapacity) {
        std::vector<std::unique_ptr<Node>> fresh(newCapacity);
        for (auto& head : buckets_) {
            while (head) {
                std::unique_ptr<Node> n = std::move(head);
                head = std::move(n->next);
                std::unique_ptr<Node>& dest = fresh[homeIndex(n->hash, newCapacity)];
                n->next = std::move(dest);
                dest = std::move(n);
            }
        }
        const std::size_t oldCapacity = buckets_.size();
        buckets_.swap(fresh);
        notifyRehash(oldCapacity, newCapacity);
    }

    // Iterative teardown so a very long chain can't overflow the stack through
    // recursive unique_ptr destructors.
    void clear() {
        for (auto& head : buckets_) {
            while (head) head = std::move(head->next);
        }
        size_ = 0;
    }

    std::vector<std::unique_ptr<Node>> buckets_;
    std::size_t size_ = 0;
    double maxLoad_;
};

// ============================================================================
// Open addressing: linear, quadratic, double hashing
// ============================================================================

// Open addressing: every entry lives directly in the slot array. On a
// collision we probe a sequence of other slots. Three probe sequences:
//
//   Linear     : home + i                (mod capacity)
//   Quadratic  : home + i(i+1)/2         (triangular numbers; with a
//                                         power-of-two capacity this visits
//                                         every slot exactly once)
//   DoubleHash : home + i * step(h)      (step is odd, so it is coprime with
//                                         a power-of-two capacity)
//
// Deletion uses tombstones: a removed slot is marked Deleted rather than
// Empty, because an Empty slot would cut the probe chain and hide keys stored
// further along it. Tombstones are reused by later inserts and flushed by
// rehashing.



enum class ProbeMode { Linear, Quadratic, DoubleHash };

class OpenAddressingTable : public UserTable {
public:
    explicit OpenAddressingTable(ProbeMode mode, std::size_t initialCapacity = 8,
                                 double maxLoad = 0.5)
        : slots_(roundUpPow2(initialCapacity)), maxLoad_(maxLoad), mode_(mode) {}

    bool insert(const std::string& username, const UserRecord& record) override {
        const uint64_t h = demoHash(username);
        const std::size_t existing = locate(h, username, /*countProbes=*/false);
        if (existing != npos) {
            slots_[existing].value = record;
            return false;
```

```
        }
        // Tombstones lengthen probe sequences just like live entries, so they
        // count toward the resize threshold.
        if (static_cast<double>(size_ + tombstones_ + 1) / slots_.size() > maxLoad_) {
            // If live entries alone are past half the threshold, grow.
            // Otherwise the table is mostly tombstones: rebuild at the same size.
            const bool grow = static_cast<double>(size_ + 1) / slots_.size() > maxLoad_ / 2;
            rehash(grow ? slots_.size() * 2 : slots_.size());
        }
        place(h, username, record);
        ++size_;
        return true;
    }

    const UserRecord* find(const std::string& username) const override {
        ++stats_.lookups;
        const std::size_t idx = locate(demoHash(username), username, /*countProbes=*/true);
        return idx == npos ? nullptr : &slots_[idx].value;
    }

    bool remove(const std::string& username) override {
        const std::size_t idx = locate(demoHash(username), username, /*countProbes=*/false);
        if (idx == npos) return false;
        Slot& s = slots_[idx];
        s.state = State::Deleted;  // tombstone keeps the probe chain intact
        s.key.clear();
        s.value = UserRecord{};
        --size_;
        ++tombstones_;
        return true;
    }

    double loadFactor() const override {
        return static_cast<double>(size_) / slots_.size();
    }
    std::size_t size() const override { return size_; }
    std::size_t capacity() const override { return slots_.size(); }
    std::size_t tombstones() const { return tombstones_; }

    std::string strategyName() const override {
        switch (mode_) {
            case ProbeMode::Linear: return "linear probing";
            case ProbeMode::Quadratic: return "quadratic probing";
            case ProbeMode::DoubleHash: return "double hashing";
        }
        return "open addressing";
    }

private:
    enum class State : uint8_t { Empty, Occupied, Deleted };
    struct Slot {
        State state = State::Empty;
        uint64_t hash = 0;
        std::string key;
        UserRecord value;
    };
    static constexpr std::size_t npos = static_cast<std::size_t>(-1);

    // i-th slot in the probe sequence for hash h.
    std::size_t probeAt(uint64_t h, std::size_t i, std::size_t cap) const {
        const std::size_t mask = cap - 1;
        const std::size_t home = homeIndex(h, cap);
        switch (mode_) {
            case ProbeMode::Linear: return (home + i) & mask;
            case ProbeMode::Quadratic: return (home + i * (i + 1) / 2) & mask;
            case ProbeMode::DoubleHash: return (home + i * secondaryStep(h)) & mask;
        }
        return home;
    }

    // Index of the slot holding key, or npos. Stops at the first Empty slot.
    std::size_t locate(uint64_t h, const std::string& key, bool countProbes) const {
        const std::size_t cap = slots_.size();
        for (std::size_t i = 0; i < cap; ++i) {
            const std::size_t idx = probeAt(h, i, cap);
            if (countProbes) ++stats_.probes;
            const Slot& s = slots_[idx];
            if (s.state == State::Empty) return npos;
            if (s.state == State::Occupied && s.hash == h && s.key == key) return idx;
        }
```

```
        return npos;
    }

    // Put a key known to be absent into the first Empty or Deleted slot.
    void place(uint64_t h, const std::string& key, const UserRecord& record) {
        const std::size_t cap = slots_.size();
        for (std::size_t i = 0; i < cap; ++i) {
            Slot& s = slots_[probeAt(h, i, cap)];
            if (s.state != State::Occupied) {
                if (s.state == State::Deleted) --tombstones_;
                s.state = State::Occupied;
                s.hash = h;
                s.key = key;
                s.value = record;
                return;
            }
        }
        // Unreachable while maxLoad < 1: a free slot always exists.
    }

    void rehash(std::size_t newCapacity) {
        std::vector<Slot> old(newCapacity);
        old.swap(slots_);  // slots_ is now an empty array of the new size
        tombstones_ = 0;
        for (Slot& s : old) {
            if (s.state == State::Occupied) place(s.hash, s.key, s.value);
        }
        notifyRehash(old.size(), newCapacity);
    }

    std::vector<Slot> slots_;
    std::size_t size_ = 0;
    std::size_t tombstones_ = 0;
    double maxLoad_;
    ProbeMode mode_;
};

// ============================================================================
// Strategy factory
// ============================================================================

enum class Strategy { SeparateChaining, LinearProbing, QuadraticProbing, DoubleHashing };

constexpr Strategy kAllStrategies[] = {Strategy::SeparateChaining, Strategy::LinearProbing,
                                       Strategy::QuadraticProbing, Strategy::DoubleHashing};

// Default max load factors: chaining tolerates high load gracefully;
// open addressing degrades quickly past ~0.5-0.7, so it resizes earlier.
inline double defaultMaxLoad(Strategy s) {
    return s == Strategy::SeparateChaining ? 0.75 : 0.5;
}

// maxLoad <= 0 means "use the default for this strategy".
inline std::unique_ptr<UserTable> makeTable(Strategy s, std::size_t initialCapacity = 8,
                                            double maxLoad = 0.0) {
    if (maxLoad <= 0.0) maxLoad = defaultMaxLoad(s);
    switch (s) {
        case Strategy::SeparateChaining:
            return std::make_unique<HashTable>(initialCapacity, maxLoad);
        case Strategy::LinearProbing:
            return std::make_unique<OpenAddressingTable>(ProbeMode::Linear, initialCapacity, maxLoad);
        case Strategy::QuadraticProbing:
            return std::make_unique<OpenAddressingTable>(ProbeMode::Quadratic, initialCapacity, maxLoad);
        case Strategy::DoubleHashing:
            return std::make_unique<OpenAddressingTable>(ProbeMode::DoubleHash, initialCapacity, maxLoad);
    }
    return nullptr;
}

// ============================================================================
// Authentication service
// ============================================================================

enum class LoginResult { Success, WrongPassword, UnknownUser };

const char* loginResultName(LoginResult r);

class AuthService {
public:
    // The table is injected so the same login code runs on every strategy.
```

```
    // saltSeed makes the demo output reproducible for the write-up; a real
    // system would draw salts from a cryptographically secure source.
    AuthService(std::unique_ptr<UserTable> table, uint64_t saltSeed);

    // Returns false (and changes nothing) if the username is already taken.
    bool registerUser(const std::string& username, const std::string& password);
    LoginResult login(const std::string& username, const std::string& password) const;
    bool removeUser(const std::string& username);

    const UserTable& table() const { return *users_; }

    // demoHash(salt + password + pepper)
    static uint64_t hashPassword(const std::string& salt, const std::string& password);

private:
    std::string makeSalt();

    std::unique_ptr<UserTable> users_;
    std::mt19937_64 rng_;
};

// ============================================================================
// Authentication service implementation
// ============================================================================

namespace {
// The pepper is a secret shared by every account and kept OUT of the user
// table. If the table leaks but the pepper does not, stored hashes cannot be
// checked offline. In production it would come from a secrets manager or
// environment variable, never source code; it is hard-coded here for the demo.
const std::string kPepper = "CS210-pepper-v1";
}  // namespace

const char* loginResultName(LoginResult r) {
    switch (r) {
        case LoginResult::Success: return "SUCCESS";
        case LoginResult::WrongPassword: return "WRONG PASSWORD";
        case LoginResult::UnknownUser: return "UNKNOWN USER";
    }
    return "?";
}

AuthService::AuthService(std::unique_ptr<UserTable> table, uint64_t saltSeed)
    : users_(std::move(table)), rng_(saltSeed) {}

uint64_t AuthService::hashPassword(const std::string& salt, const std::string& password) {
    return demoHash(salt + password + kPepper);
}

std::string AuthService::makeSalt() {
    return toHex(rng_());  // 64 random bits -> 16 hex chars
}

bool AuthService::registerUser(const std::string& username, const std::string& password) {
    if (users_->find(username) != nullptr) return false;
    UserRecord rec;
    rec.username = username;
    rec.salt = makeSalt();
    rec.passwordHash = hashPassword(rec.salt, password);
    users_->insert(username, rec);
    return true;
}

LoginResult AuthService::login(const std::string& username, const std::string& password) const {
    const UserRecord* rec = users_->find(username);  // expected O(1)
    if (rec == nullptr) return LoginResult::UnknownUser;
    // Recompute with the stored salt and compare. (Real systems also use a
    // constant-time comparison; one 64-bit compare is already branch-free.)
    return hashPassword(rec->salt, password) == rec->passwordHash ? LoginResult::Success
                                                                  : LoginResult::WrongPassword;
}

bool AuthService::removeUser(const std::string& username) {
    return users_->remove(username);
}

// ============================================================================
// DEMO MODE (default): authentication walkthrough
// ============================================================================
```

```
namespace demo {

// Demonstrates: registration of salted+peppered users, successful login,
// failed password, unknown user, removal, rehash events, visible collisions,
// and the same flow running on all four collision strategies.


namespace {

int g_pass = 0;
int g_fail = 0;

void check(bool ok, const std::string& what) {
    std::cout << (ok ? "    [PASS] " : "    [FAIL] ") << what << '\n';
    (ok ? g_pass : g_fail)++;
}

void section(const std::string& title) {
    std::cout << "\n=== " << title << " ===\n";
}

void attempt(const AuthService& auth, const std::string& user, const std::string& pw,
             LoginResult expected) {
    const LoginResult r = auth.login(user, pw);
    std::cout << "  login(\"" << user << "\", \"" << pw << "\") -> " << loginResultName(r) << '\n';
    check(r == expected, std::string("expected ") + loginResultName(expected));
}

void printTableState(const UserTable& t) {
    std::cout << "    size=" << t.size() << " capacity=" << t.capacity() << " load="
              << std::fixed << std::setprecision(3) << t.loadFactor() << '\n';
}

void printBuckets(const HashTable& t) {
    for (std::size_t b = 0; b < t.capacity(); ++b) {
        const auto keys = t.bucketKeys(b);
        std::cout << "    [" << std::setw(2) << b << "] ";
        if (keys.empty()) std::cout << "(empty)";
        for (std::size_t i = 0; i < keys.size(); ++i) std::cout << (i ? " -> " : "") << keys[i];
        if (keys.size() > 1) std::cout << "   <-- collision, chain length " << keys.size();
        std::cout << '\n';
    }
}

struct DemoUser {
    std::string name;
    std::string password;
};

const std::vector<DemoUser> kUsers = {
    {"alice", "Wonderland#42"}, {"bob", "b0bcat!"},       {"carmen", "SanDiego2026"},
    {"dmitri", "Tchaikovsky1840"}, {"eunji", "kimchi&rice"}, {"farah", "desertRose7"},
    {"gabe", "hunter2"},        {"hiro", "hunter2"},  // same password on purpose
};

constexpr uint64_t kSaltSeed = 210;

}  // namespace

int runDemo() {
    std::cout << "CS 210 Assignment 06 - Passwords and Hash Tables (Talan Jewett)\n";

    auto chained = std::make_unique<HashTable>(4, 0.75);
    HashTable* table = chained.get();  // non-owning view for bucket printing
    table->setRehashListener([](const std::string& s, std::size_t oldCap, std::size_t newCap,
                                std::size_t moved) {
        std::cout << "  [REHASH] " << s << ": " << oldCap << " -> " << newCap << " buckets ("
                  << moved << " entries relinked)\n";
    });
    AuthService auth(std::move(chained), kSaltSeed);

    // ------------------------------------------------------------------
    section("1. Register 8 users (separate chaining, start capacity 4, max load 0.75)");
    for (const auto& u : kUsers) {
        auth.registerUser(u.name, u.password);
        const UserRecord* r = table->find(u.name);
        std::cout << "  + " << std::left << std::setw(7) << u.name << std::right
                  << " salt=" << r->salt << "  hash=" << toHex(r->passwordHash)
                  << "  load=" << std::fixed << std::setprecision(3) << table->loadFactor()
                  << " (" << table->size() << "/" << table->capacity() << ")\n";
```

```
}
check(table->size() == 8, "8 users stored");
check(table->stats().rehashes == 2, "two automatic rehash events (4->8, 8->16)");
check(table->loadFactor() <= 0.75, "load factor stays <= 0.75");
check(!auth.registerUser("alice", "different"), "duplicate username is rejected");

// ------------------------------------------------------------------
section("2. Bucket layout (collisions handled by chaining)");
printBuckets(*table);
std::cout << "    longest chain = " << table->longestChain() << '\n';

// ------------------------------------------------------------------
section("3. Salt demonstration: gabe and hiro share the password \"hunter2\"");
const UserRecord* g = table->find("gabe");
const UserRecord* h = table->find("hiro");
std::cout << "  gabe: salt=" << g->salt << " hash=" << toHex(g->passwordHash) << '\n';
std::cout << "  hiro: salt=" << h->salt << " hash=" << toHex(h->passwordHash) << '\n';
check(g->salt != h->salt, "salts differ");
check(g->passwordHash != h->passwordHash, "same password -> different stored hashes");
check(AuthService::hashPassword(g->salt, "hunter2") == g->passwordHash,
      "hash recomputes from stored salt + password + pepper");

// ------------------------------------------------------------------
section("4. Login flow");
attempt(auth, "alice", "Wonderland#42", LoginResult::Success);
attempt(auth, "carmen", "SanDiego2026", LoginResult::Success);
attempt(auth, "alice", "wonderland#42", LoginResult::WrongPassword);  // case differs
attempt(auth, "dmitri", "", LoginResult::WrongPassword);
attempt(auth, "gabe", "hunter3", LoginResult::WrongPassword);
attempt(auth, "mallory", "Wonderland#42", LoginResult::UnknownUser);
attempt(auth, "Alice", "Wonderland#42", LoginResult::UnknownUser);  // keys are case-sensitive

// ------------------------------------------------------------------
section("5. Removal");
check(auth.removeUser("bob"), "remove(\"bob\") returns true");
attempt(auth, "bob", "b0bcat!", LoginResult::UnknownUser);
check(!auth.removeUser("bob"), "removing bob again returns false");
check(table->find("bob") == nullptr, "find(\"bob\") returns nullptr");
check(table->size() == 7, "size is now 7");
attempt(auth, "eunji", "kimchi&rice", LoginResult::Success);  // neighbours unaffected
printTableState(*table);

// ------------------------------------------------------------------
section("6. Rehash event: add users until load would exceed 0.75");
const std::size_t before = table->capacity();
for (int i = 1; i <= 6; ++i) {
    const std::string name = "student" + std::to_string(i);
    auth.registerUser(name, "pw" + std::to_string(i));
    std::cout << "  + " << name << "  load=" << std::fixed << std::setprecision(3)
              << table->loadFactor() << " (" << table->size() << "/" << table->capacity()
              << ")\n";
}
check(table->capacity() == before * 2, "capacity doubled from 16 to 32");
check(table->stats().rehashes == 3, "third rehash event recorded");
bool allOk = true;
for (const auto& u : kUsers) {
    if (u.name == "bob") continue;
    allOk = allOk && auth.login(u.name, u.password) == LoginResult::Success;
}
check(allOk, "every original user still logs in after the rehash");
attempt(auth, "student6", "pw6", LoginResult::Success);

// ------------------------------------------------------------------
section("7. Same workflow on all four collision strategies");
std::cout << "  " << std::left << std::setw(20) << "strategy" << std::right << std::setw(6)
          << "size" << std::setw(6) << "cap" << std::setw(8) << "load" << std::setw(10)
          << "rehashes" << std::setw(12) << "avg probes" << "  logins\n";
for (Strategy s : kAllStrategies) {
    AuthService a(makeTable(s, 4), kSaltSeed);
    for (const auto& u : kUsers) a.registerUser(u.name, u.password);
    a.removeUser("bob");
    for (int i = 1; i <= 6; ++i) a.registerUser("student" + std::to_string(i), "pw" + std::to_string(i));
    a.table().resetProbeStats();
    bool ok = a.login("bob", "b0bcat!") == LoginResult::UnknownUser &&
              a.login("alice", "nope") == LoginResult::WrongPassword;
    for (const auto& u : kUsers)
        if (u.name != "bob") ok = ok && a.login(u.name, u.password) == LoginResult::Success;
    const UserTable& t = a.table();
    std::cout << "  " << std::left << std::setw(20) << t.strategyName() << std::right
              << std::setw(6) << t.size() << std::setw(6) << t.capacity() << std::setw(8)
```

```
                << std::setprecision(3) << t.loadFactor() << std::setw(10)
                << t.stats().rehashes << std::setw(12) << std::setprecision(2)
                << t.stats().avgProbes() << "  " << (ok ? "all correct" : "MISMATCH") << '\n';
        check(ok, t.strategyName() + ": success / wrong password / unknown user all correct");
    }

    std::cout << "\nSummary: " << g_pass << " checks passed, " << g_fail << " failed\n";
    return g_fail == 0 ? 0 : 1;
}

}  // namespace demo

// ============================================================================
// TEST MODE: unit, edge-case and randomized tests
// ============================================================================

namespace tests {

// Every table test runs against all four strategies. std::map is used ONLY as
// a reference oracle in the randomized test; it never stores user records.


namespace {

int g_pass = 0;
int g_fail = 0;

#define EXPECT(cond)                                                          \
    do {                                                                      \
        if (cond) {                                                           \
            ++g_pass;                                                         \
        } else {                                                              \
            ++g_fail;                                                         \
            std::cout << "    FAIL line " << __LINE__ << ": " #cond << '\n';  \
        }                                                                     \
    } while (0)

UserRecord rec(const std::string& user, uint64_t h = 0) {
    UserRecord r;
    r.username = user;
    r.salt = "salt";
    r.passwordHash = h;
    return r;
}

bool isPow2(std::size_t n) { return n && !(n & (n - 1)); }

void testHashFunction() {
    // Regression values for the course-provided demoHash. Note: its offset
    // basis (1469598103934665603) is one digit shorter than standard FNV-1a's
    // (14695981039346656037), so outputs differ from published FNV-1a vectors.
    // I use the function exactly as provided.
    EXPECT(demoHash("") == 14695981039346656037ULL);
    EXPECT(demoHash("a") == 0x44bd8ad473cd9906ULL);
    EXPECT(demoHash("foobar") == 0x88fad7c0a8ff07f2ULL);
    EXPECT(demoHash("alice") == demoHash("alice"));  // deterministic
    EXPECT(demoHash("alice") != demoHash("Alice"));
}

void testBasics(Strategy s) {
    auto t = makeTable(s, 8);
    EXPECT(t->size() == 0);
    EXPECT(t->loadFactor() == 0.0);
    EXPECT(t->find("ghost") == nullptr);
    EXPECT(!t->remove("ghost"));

    EXPECT(t->insert("alice", rec("alice", 1)));
    EXPECT(!t->insert("alice", rec("alice", 2)));  // duplicate key -> update, not a new entry
    EXPECT(t->size() == 1);
    const UserRecord* r = t->find("alice");
    EXPECT(r != nullptr && r->passwordHash == 2);

    EXPECT(t->insert("bob", rec("bob", 3)));
    EXPECT(std::fabs(t->loadFactor() - 2.0 / t->capacity()) < 1e-12);

    EXPECT(t->remove("alice"));
    EXPECT(t->find("alice") == nullptr);
    EXPECT(!t->remove("alice"));
    EXPECT(t->size() == 1);
    EXPECT(t->find("bob") != nullptr);
```

```
    EXPECT(t->insert("", rec("", 9)));  // empty string is a legal key
    EXPECT(t->find("") != nullptr && t->find("")->passwordHash == 9);
}

void testResize(Strategy s) {
    auto t = makeTable(s, 2);
    const double maxLoad = defaultMaxLoad(s);
    bool loadOk = true;
    for (int i = 0; i < 5000; ++i) {
        t->insert("user" + std::to_string(i), rec("u", static_cast<uint64_t>(i)));
        loadOk = loadOk && t->loadFactor() <= maxLoad;
    }
    EXPECT(loadOk);
    EXPECT(t->size() == 5000);
    EXPECT(isPow2(t->capacity()));
    EXPECT(t->stats().rehashes >= 10);
    bool allFound = true;
    for (int i = 0; i < 5000; ++i) {
        const UserRecord* r = t->find("user" + std::to_string(i));
        allFound = allFound && r && r->passwordHash == static_cast<uint64_t>(i);
    }
    EXPECT(allFound);
}

void testForcedCollisions(Strategy s) {
    const std::size_t cap = 16;
    // Very high max load for chaining so the table never resizes in this test.
    auto t = makeTable(s, cap, s == Strategy::SeparateChaining ? 100.0 : 0.95);
    std::vector<std::string> keys;  // six keys that all hash to bucket 3
    for (int i = 0; keys.size() < 6; ++i) {
        const std::string k = "k" + std::to_string(i);
        if (homeIndex(demoHash(k), cap) == 3) keys.push_back(k);
    }
    for (const auto& k : keys) t->insert(k, rec(k));
    EXPECT(t->capacity() == cap);
    bool allFound = true;
    for (const auto& k : keys) allFound = allFound && t->find(k) != nullptr;
    EXPECT(allFound);

    // Removing from the middle of a collision cluster must not hide later keys
    // (this is what tombstones are for in open addressing).
    EXPECT(t->remove(keys[2]));
    EXPECT(t->find(keys[2]) == nullptr);
    bool othersFound = true;
    for (std::size_t i = 0; i < keys.size(); ++i)
        if (i != 2) othersFound = othersFound && t->find(keys[i]) != nullptr;
    EXPECT(othersFound);
    EXPECT(t->insert(keys[2], rec(keys[2])));  // reinsert (reuses tombstone)
    EXPECT(t->find(keys[2]) != nullptr);
    EXPECT(t->size() == keys.size());
}

void testTombstoneChurn(Strategy s) {
    // Insert/remove cycles must not grow the table forever or loop infinitely.
    auto t = makeTable(s, 8);
    t->insert("keepA", rec("keepA"));
    t->insert("keepB", rec("keepB"));
    for (int i = 0; i < 20000; ++i) {
        const std::string k = "tmp" + std::to_string(i);
        t->insert(k, rec(k));
        t->remove(k);
    }
    EXPECT(t->size() == 2);
    EXPECT(t->capacity() <= 16);
    EXPECT(t->find("keepA") != nullptr && t->find("keepB") != nullptr);
}

void testRandomizedAgainstReference(Strategy s) {
    auto t = makeTable(s, 4);
    std::map<std::string, uint64_t> ref;  // oracle only
    std::mt19937 rng(42);
    std::uniform_int_distribution<int> keyDist(0, 1999), opDist(0, 2);
    bool consistent = true;
    for (int step = 0; step < 50000; ++step) {
        const std::string k = "key" + std::to_string(keyDist(rng));
        const uint64_t v = static_cast<uint64_t>(step);
        switch (opDist(rng)) {
            case 0: {
                const bool isNew = ref.find(k) == ref.end();
                consistent = consistent && t->insert(k, rec(k, v)) == isNew;
```

```
                ref[k] = v;
                break;
            }
            case 1:
                consistent = consistent && t->remove(k) == (ref.erase(k) == 1);
                break;
            default: {
                const UserRecord* r = t->find(k);
                auto it = ref.find(k);
                consistent = consistent && (it == ref.end() ? r == nullptr
                                                            : r && r->passwordHash == it->second);
            }
        }
    }
    EXPECT(consistent);
    EXPECT(t->size() == ref.size());
    bool finalMatch = true;
    for (const auto& kv : ref) {
        const UserRecord* r = t->find(kv.first);
        finalMatch = finalMatch && r && r->passwordHash == kv.second;
    }
    EXPECT(finalMatch);
}

void testAuth(Strategy s) {
    AuthService auth(makeTable(s, 2), 7);
    const std::vector<std::pair<std::string, std::string>> users = {
        {"ana", "pa55"}, {"ben", "secret"}, {"cy", "secret"}, {"dee", "x"}, {"eli", "!@#$"}};
    for (const auto& u : users) EXPECT(auth.registerUser(u.first, u.second));
    EXPECT(!auth.registerUser("ana", "other"));
    EXPECT(auth.table().size() == 5);
    for (const auto& u : users) EXPECT(auth.login(u.first, u.second) == LoginResult::Success);
    EXPECT(auth.login("ana", "PA55") == LoginResult::WrongPassword);
    EXPECT(auth.login("ana", "") == LoginResult::WrongPassword);
    EXPECT(auth.login("zed", "pa55") == LoginResult::UnknownUser);
    EXPECT(auth.login("", "") == LoginResult::UnknownUser);
    const UserRecord* b = auth.table().find("ben");
    const UserRecord* c = auth.table().find("cy");
    EXPECT(b->salt.size() == 16 && b->salt != c->salt);
    EXPECT(b->passwordHash != c->passwordHash);  // same password, different salts
    EXPECT(auth.removeUser("ben"));
    EXPECT(auth.login("ben", "secret") == LoginResult::UnknownUser);
    EXPECT(auth.login("cy", "secret") == LoginResult::Success);
}

void runGroup(const std::string& name, void (*fn)(Strategy)) {
    for (Strategy s : kAllStrategies) {
        const int failBefore = g_fail;
        fn(s);
        std::cout << "  " << (g_fail == failBefore ? "ok  " : "FAIL") << "  " << name << " ["
                  << makeTable(s)->strategyName() << "]\n";
    }
}

}  // namespace

int runTests() {
    std::cout << "Running hash table tests\n";
    testHashFunction();
    std::cout << "  " << (g_fail ? "FAIL" : "ok  ") << "  demoHash reference values\n";
    runGroup("insert/find/remove/loadFactor basics", testBasics);
    runGroup("automatic resize keeps all keys (5000 inserts)", testResize);
    runGroup("forced collisions + removal mid-cluster", testForcedCollisions);
    runGroup("tombstone churn (20000 insert/remove cycles)", testTombstoneChurn);
    runGroup("50000 random ops vs reference map", testRandomizedAgainstReference);
    runGroup("auth: register/login/remove", testAuth);
    std::cout << "\n" << g_pass << " assertions passed, " << g_fail << " failed\n";
    return g_fail == 0 ? 0 : 1;
}

}  // namespace tests

// ============================================================================
// BENCH MODE: compare the four collision strategies
// ============================================================================

namespace bench {

// Part A: average probes per lookup at fixed load factors (no resizing).
// Part B: wall-clock time for a realistic workload with automatic resizing
```

```
//         and a delete-heavy phase.


namespace {

UserRecord rec(const std::string& u) {
    UserRecord r;
    r.username = u;
    return r;
}

std::vector<std::string> makeKeys(const std::string& prefix, std::size_t n) {
    std::vector<std::string> v;
    v.reserve(n);
    for (std::size_t i = 0; i < n; ++i) v.push_back(prefix + std::to_string(i));
    return v;
}

double msSince(std::chrono::steady_clock::time_point t0) {
    return std::chrono::duration<double, std::milli>(std::chrono::steady_clock::now() - t0).count();
}

}  // namespace

int runBench() {
    const std::size_t cap = 1u << 16;  // 65,536 slots
    const auto hitKeys = makeKeys("user", cap);
    const auto missKeys = makeKeys("miss", cap);

    std::cout << "Part A: average probes per lookup, capacity " << cap << ", no resizing\n";
    std::cout << std::left << std::setw(20) << "strategy" << std::right;
    const double loads[] = {0.25, 0.50, 0.75, 0.90};
    for (double a : loads) std::cout << "   hit@" << std::fixed << std::setprecision(2) << a << "  miss@" << a;
    std::cout << '\n';

    for (Strategy s : kAllStrategies) {
        std::cout << std::left << std::setw(20) << makeTable(s)->strategyName() << std::right;
        for (double a : loads) {
            auto t = makeTable(s, cap, 0.99);
            const std::size_t n = static_cast<std::size_t>(a * cap);
            for (std::size_t i = 0; i < n; ++i) t->insert(hitKeys[i], rec(hitKeys[i]));
            t->resetProbeStats();
            for (std::size_t i = 0; i < n; ++i) t->find(hitKeys[i]);
            const double hit = t->stats().avgProbes();
            t->resetProbeStats();
            for (std::size_t i = 0; i < n; ++i) t->find(missKeys[i]);
            const double miss = t->stats().avgProbes();
            std::cout << std::setprecision(2) << std::setw(10) << hit << std::setw(11) << miss;
        }
        std::cout << '\n';
    }

    const std::size_t N = 200000;
    const auto keys = makeKeys("member", N);
    const auto more = makeKeys("newbie", N / 2);
    const auto misses = makeKeys("nobody", N);
    std::cout << "\nPart B: " << N << " users, default max load, start capacity 8 (ms, best of 3)\n";
    std::cout << std::left << std::setw(20) << "strategy" << std::right << std::setw(10) << "insert"
              << std::setw(10) << "find hit" << std::setw(11) << "find miss" << std::setw(13)
              << "remove 50%" << std::setw(13) << "re-insert" << std::setw(10) << "cap"
              << std::setw(10) << "rehashes" << '\n';
    for (Strategy s : kAllStrategies) {
        double best[5] = {1e18, 1e18, 1e18, 1e18, 1e18};
        std::size_t finalCap = 0;
        uint64_t rehashes = 0;
        for (int rep = 0; rep < 3; ++rep) {
            auto t = makeTable(s, 8);
            double ms[5];
            auto t0 = std::chrono::steady_clock::now();
            for (const auto& k : keys) t->insert(k, rec(k));
            ms[0] = msSince(t0);
            t0 = std::chrono::steady_clock::now();
            for (const auto& k : keys) t->find(k);
            ms[1] = msSince(t0);
            t0 = std::chrono::steady_clock::now();
            for (const auto& k : misses) t->find(k);
            ms[2] = msSince(t0);
            t0 = std::chrono::steady_clock::now();
            for (std::size_t i = 0; i < N; i += 2) t->remove(keys[i]);
            ms[3] = msSince(t0);
```

```
            t0 = std::chrono::steady_clock::now();
            for (const auto& k : more) t->insert(k, rec(k));
            ms[4] = msSince(t0);
            for (int i = 0; i < 5; ++i) best[i] = ms[i] < best[i] ? ms[i] : best[i];
            finalCap = t->capacity();
            rehashes = t->stats().rehashes;
        }
        std::cout << std::left << std::setw(20) << makeTable(s)->strategyName() << std::right
                  << std::setprecision(1) << std::setw(10) << best[0] << std::setw(10) << best[1]
                  << std::setw(11) << best[2] << std::setw(13) << best[3] << std::setw(13)
                  << best[4] << std::setw(10) << finalCap << std::setw(10) << rehashes << '\n';
    }
    return 0;
}

}  // namespace bench

// ============================================================================
// Entry point
// ============================================================================

int main(int argc, char* argv[]) {
    const std::string mode = argc > 1 ? argv[1] : "demo";
    if (mode == "demo") return demo::runDemo();
    if (mode == "test") return tests::runTests();
    if (mode == "bench") return bench::runBench();
    std::cerr << "usage: " << argv[0] << " [demo|test|bench]\n";
    return 2;
}
```